Plan Review: Agentic Quality Inspection System

Review of the submitted architecture diagram against the agreed 10-day plan for the OpenCV AI Competition 2026 (deadline: October 26, 2026, 11:59 p.m. PT).

1. Your Diagram (as submitted)

```
Input image (MVTec AD image or uploaded test image)
        ↓
Perception — PatchScorer (anomaly score, heatmap, candidate ROIs)
        ↓
Agent planner and budget controller (threshold policy + evidence-driven tool selection)
        ↓
OpenCV 5 tool executor (Crop · CLAHE · Sharpen · Next ROI)
        ↻
Re-perception and evidence comparison (repeat until decisive or budget exhausted)
        ↓
PASS  |  FAIL  |  HUMAN
```

2. Overall Verdict

The core idea is right. It has a real closed loop, a budget limit, and three outcomes including human review. That is exactly what the Agentic Vision rule asks for: the visual result must change the next action (OpenCV rules).

But it has 3 technical problems and 5 missing pieces that judges will notice. All are fixable within the 10 days.

Area	Status
Closed perception → decision → action loop	✅ Matches plan
Budget controller (step/time limit)	✅ Good, and better than the plan (supports the observability and failure-handling scores)
PASS / FAIL / HUMAN outcomes	✅ Matches plan
Evidence-driven tool selection	✅ Matches plan
Image quality gate before perception	❌ Missing
Golden-template (reference) comparison tool	❌ Missing (this is the strongest tool for MVTec objects)
Defect measurement / explanation output	❌ Missing (the project title says “explainable”)
AWS + logging in the diagram	❌ Missing (the rules require the architecture diagram to show OpenCV 5 and AWS components)
Clear rules for “decisive” and FAIL vs HUMAN	⚠️ Not defined
CLAHE / Sharpen before re-scoring	⚠️ Technical risk (see 3.1)
Re-scoring a zoomed crop with the same model	⚠️ Technical risk (see 3.2)

3. Technical Problems to Fix

3.1 CLAHE and Sharpen can create fake anomalies

PatchCore learns what "normal" looks like from **unprocessed** training images. If you apply CLAHE or sharpening to a test image and score it again, the textures change, and the model may flag the image **because it was processed**, not because it is defective. The loop could then push the score up on every pass and turn good parts into FAILs.

Fix (choose one): - **Option A (recommended):** Use CLAHE/Sharpen only on images that **you** look at (visualization) and for measurement (contours). Never feed them back into PatchCore. - **Option B:** Build a second PatchCore memory bank from normal training images processed with exactly the same CLAHE settings, and score processed images only against that bank.

3.2 A zoomed crop is not comparable to the full image

PatchCore at 256×256 sees a full bottle. A crop resized to 256×256 shows the surface at a very different scale. Its patch features won't match the full-image memory bank, so the crop score will be high even on a perfect part.

Fix: - Crop the **same region from an aligned defect-free reference image**, and compare the two crops (structural difference or patch-feature distance). This turns "zoom" into a fair comparison. - Or build a separate "crop-scale" memory bank from random crops of normal training images.

3.3 The loop needs a clear "decisive" rule

"Repeat until decisive" must be a concrete rule, or judges cannot assess it and the loop may run without end.

Proposed rules (tune them on a calibration split):

Condition	Action
Score < τ_low	**PASS**
Score ≥ τ_high **and** reference comparison confirms the defect	**FAIL**
τ_low ≤ score < τ_high	Run the next tool (zoom + reference comparison on the top ROI)
Two pieces of evidence disagree (e.g., heatmap high but reference difference low)	Run the next ROI, or escalate
Budget used up (e.g., 3 steps or 1.5 s) and still uncertain	**HUMAN**
Image fails the quality gate	**HUMAN / RETAKE** (no score is produced)

4. Missing Pieces to Add

4.1 Image quality gate (before PatchScorer)

- Blur: `cv.Laplacian` variance
- Exposure/glare: `cv.calcHist` percentiles
- If the image fails, stop and return **RETAKE / HUMAN**. This shows judges a clear failure-handling path (15% of the Agentic Vision rubric) (OpenCV rules).

4.2 Golden-template comparison tool

- Find the nearest defect-free training image for the category.
- Align it to the test image: ORB (or ALIKED) + `cv.findHomography` with RANSAC + `cv.warpPerspective`.
- Compute the masked difference and compare it with the heatmap.
- This is the "Comparative Retriever" idea from AgentIAD, done with classic OpenCV.

4.3 Measurement and explanation

- Threshold the heatmap → `cv.findContours` → area, bounding box, elongation (`cv.minAreaRect`), distance to edge.
- Output a short structured report: "Defect at (x, y), 2.1% of part area, elongated → likely scratch."
- Optional: send only these measured numbers to Amazon Bedrock to write a one-line explanation. The LLM must explain the measurements, not invent findings.

4.4 AWS and logging in the diagram

Add a bottom layer: **FastAPI on EC2 · images and traces in S3 · logs and latency in CloudWatch**. The rules say the architecture diagram must show both the OpenCV 5 and AWS components (OpenCV Competition).

4.5 Agent trace

Save every step as JSON: tool called, why it was called (the score or evidence that triggered it), result, and time taken. The Agentic Vision Award specifically asks for a trace showing OpenCV output changing a later decision (OpenCV Competition).

---

5. Revised Diagram

```
Input image (MVTec AD test image / upload)
        ↓
[OpenCV] Quality gate ── fail ──► RETAKE / HUMAN
        ↓ pass
[OpenCV DNN] PatchScorer (PatchCore ONNX) → score, heatmap, top-k ROIs
        ↓
Agent planner + budget controller (max 3 steps / 1.5 s)
   ├─ score < τ_low ──────────────────────────────► PASS
   ├─ score ≥ τ_high ─► reference check — agree ──► FAIL
   └─ uncertain ──► OpenCV 5 tool executor:
                      1. Align golden template (ORB + homography)
                      2. Crop same ROI from test + reference → compare
                      3. Next ROI
                    ↻ re-perception and evidence comparison
                    budget exhausted / evidence conflicts ──► HUMAN
        ↓
[OpenCV] Measurement: contours, area, shape → explainable report
        ↓
AWS: FastAPI on EC2 · S3 (images + JSON traces) · CloudWatch (logs, latency)
        ↓
React operator page: result, heatmap, step trace, override button
```

CLAHE and Sharpen stay in the system, but **only** for display and measurement, not for re-scoring.

---

6. Evaluation Checklist

Item	Why it matters
Use 3 categories: bottle, metal_nut, screw	Fits the time available
Split the MVTec **test** set into calibration (to tune τ_low / τ_high) and held-out test; report the split	Tuning thresholds on the same images you report results on is data leakage, and the rules prohibit misrepresenting results
Baseline: single-pass PatchCore at a fixed threshold	Gives the loop something to beat
Agent: the full loop	Your main result
Report: image AUROC, false-positive rate on good parts, recall on defects, **HUMAN escalation rate**, average steps, average latency	Escalation rate is essential: a system that sends every image to HUMAN can look error-free
Robustness test: add blur / gamma shift to some test images	Shows that the quality gate works
3–4 documented failure cases with images	The rules require failure cases or limitations

7. Updated 10-Day Plan

Day	Work
1	MVTec download; PatchCore for 3 categories (Anomalib) → ONNX → confirm it runs in OpenCV 5 DNN and gives the same scores as Anomalib
2	Quality gate + heatmap → ROI extraction + contour measurement
3	Golden-template alignment + ROI comparison tool
4	Agent planner, budget controller, decision rules, JSON trace
5–6	FastAPI + React operator page (result, heatmap, trace, override)
7	AWS deployment (EC2, S3, CloudWatch)
8	Calibration split, baseline vs agent evaluation, robustness test, failure cases
9–10	Technical report (7 required sections), final diagram, 5-minute video

8. Day 1 Risk to Check First

Confirm that the Anomalib PatchCore ONNX export (which includes the memory-bank nearest-neighbor search) **loads and runs in OpenCV 5's DNN module**. If an operator isn't supported: - Try `ENGINE_CLASSIC`, or `ENGINE_ORT` if your build includes ONNX Runtime (OpenCV 5). - Fallback: run only the backbone feature extractor (e.g., WideResNet50 ONNX) in OpenCV DNN, and do the nearest-neighbor search in NumPy. OpenCV still runs the model inference.

Solve this on Day 1, because everything else depends on it.